FLYRANK INTERNSHIP

BACKEND DEVELOPMENT TRACK · CAPSTONE PROJECT

Usage Metering & Billing Engine

Exactly-once metering · Quota enforcement · Integer money math
Signature-verified Stripe webhooks (test mode)

PROJECT REPORT

Submitted by

Ahsan Choudhry

Track	Backend Development
Capstone	Usage Metering & Billing Engine
Technology Stack	Python · FastAPI · SQLAlchemy · SQLite / PostgreSQL
Payments	Stripe — test mode (sandbox) only
Repository	github.com/Ahsa6117/flyrank-capstone-metering-billing
Automated Tests	77 passing
Date	29 August 2026

Table of Contents

1. Introduction
2. The Problem
3. What Was Built — The Solution
4. System Architecture
5. Implementation Highlights
6. Testing and Verification
7. A Defect Found and Fixed
8. Live Stripe Integration
9. Security and Secret Handling
10. Results Against the Requirements
11. Limitations and Future Work
12. Conclusion

1. Introduction

Every software-as-a-service product, without exception, has to answer three questions about each of its customers: how much have they used, what should they pay for it, and have they reached the limit of their plan. This report describes the backend service I built to answer all three.

The project was completed for the FlyRank Internship Backend Development Track. The brief describes it as medium difficulty, but notes that the difficulty sits in precision rather than size. That turned out to be an accurate description. The system is deliberately small — two plans, two usage types, one billable endpoint — yet almost every part of it is a place where a subtle mistake costs real money.

The service delivers four capabilities:

- **Usage metering.** Every billable action is recorded against a tenant, and a retried request is recorded exactly once.
- **Quota enforcement.** Plan limits are checked before an action is allowed, with honest and specific HTTP status codes when a request is refused.
- **Cost calculation.** Usage is converted into money using integer arithmetic, including the AI-token pricing rules where different token categories carry different rates.
- **Stripe integration.** Subscription checkout in test mode, with webhooks that verify their signature and cannot be replayed.

The finished project is a public GitHub repository containing the service, its migrations, 77 automated tests, and full documentation.

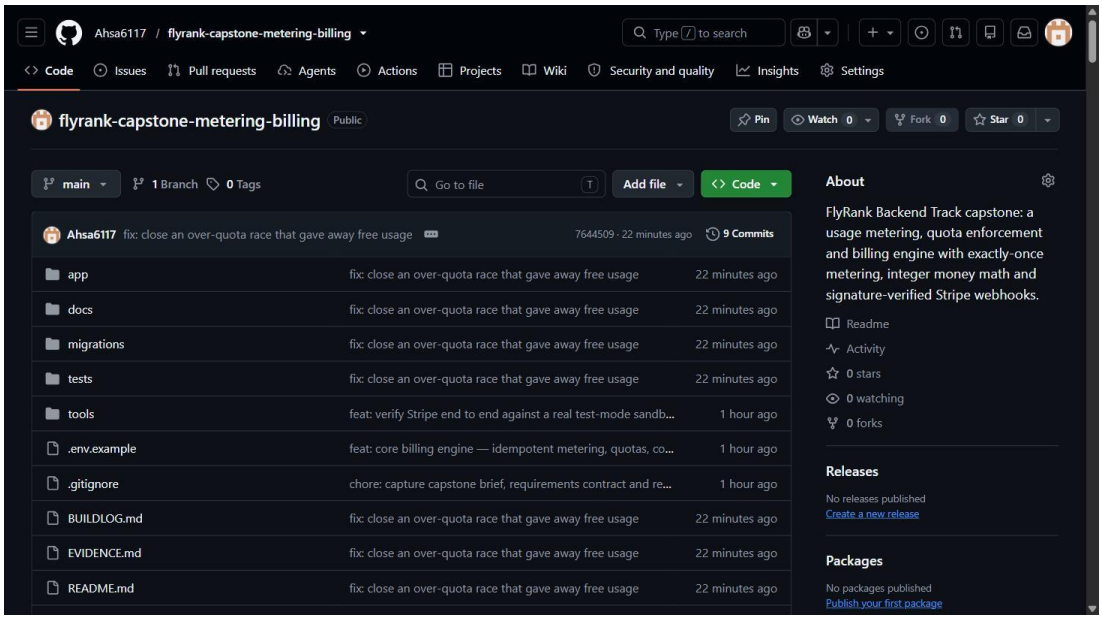

Figure 1 — The public repository, with each build phase visible in the commit history.

2. The Problem

Billing systems look straightforward from the outside: count what the customer used, multiply by a price, refuse them when they run out. The difficulty appears when the real world intrudes. A network connection drops after the server has already done the work. A payment provider delivers the same notification twice. Two requests arrive at the very instant a customer reaches their limit.

Four specific problems had to be solved.

2.1 A retried request must not be charged twice

When a client times out, it does not know whether the server processed the request. The safe behaviour is to retry. If the server treats that retry as new work, the customer is billed twice for one action. This is the failure that damages customer trust the fastest, because the customer can see it on their invoice and the provider usually cannot explain it.

2.2 A quota must hold exactly, and refuse honestly

A limit that is approximately enforced is not a limit. The brief asks the question directly: at 999 of 1,000 calls, what happens, and what happens at exactly 1,000? A customer who paid for 1,000

calls should receive precisely 1,000 — not 999, and not 1,006. When a request is refused, the response must also say why in a way both a person and a program can act on.

2.3 Money must be calculated exactly

Floating-point numbers cannot represent most decimal currency values exactly, and rounding behaviour differs between languages, so small errors accumulate silently across thousands of transactions. AI-token pricing adds a second trap: cached input tokens are cheaper than fresh input tokens, and hidden reasoning tokens are billed at the output rate. The four token categories cannot simply be added together and priced once.

2.4 Payment state must be trustworthy

The payment provider is the authority on whether a customer has paid. A service that upgrades a customer because their browser returned to a success page can be upgraded by anyone who visits that URL. Equally, a webhook endpoint that does not verify signatures can be driven by anyone who knows its address, and one that does not deduplicate will process the same event twice, because delivery is at-least-once and unordered.

3. What Was Built — The Solution

Each problem above is solved by a specific mechanism rather than by careful coding. Wherever possible the guarantee is enforced by the database, because a constraint cannot be forgotten during a later edit and does not have a race window.

Problem	Mechanism	Where it lives
Retried request billed twice	UNIQUE (tenant_id, idempotency_key); the first response is stored and replayed	migrations/001
Two requests slipping past the quota	Per-tenant write lock taken before the quota is read	migrations/002
Rounding errors in money	All amounts stored as integer micro-cents; no float anywhere	core/pricing.py
Wrong token pricing	Four counters priced separately; reasoning bound to the output rate	core/pricing.py
Forged payment notification	HMAC-SHA256 signature verified over the raw request body	api/webhooks.py
Duplicate payment notification	Deduplication on the provider's event ID	services/stripe_sync.py

3.1 Plans and quotas

Two plans were implemented. The Free tier limits are fixed by the brief; the Pro tier limits are my own choice and are documented in the repository.

Plan	API calls per month	AI tokens per month
Free	1,000	100,000
Pro	50,000	5,000,000

Quotas reset on the first day of each calendar month at midnight UTC. The token quota counts all four token categories added together, because a token consumed is a token consumed. Pricing, by contrast, keeps the categories strictly apart. That distinction is deliberate, and it is the single most easily confused point in the system, so an automated test asserts it.

3.2 Pricing constants

Prices are pinned in a single configuration module and asserted by tests, so an accidental change breaks the build rather than quietly altering an invoice.

Item	Rate	Micro-cents
API call	$0.00200 per call	2,000,000
Fresh input tokens	$0.75 per 1M	75,000,000
Cached input tokens	$0.075 per 1M	7,500,000
Output tokens	$3.75 per 1M	375,000,000
Reasoning tokens	billed at the output rate	375,000,000

4. System Architecture

The service uses a layered architecture with dependencies pointing in one direction only. This is what keeps the business rules testable without an HTTP client and the HTTP layer free of database concerns.

- **HTTP layer (app/api).** Request validation, authentication, and the single place where a domain error becomes a status code. It contains no SQL.
- **Service layer (app/services).** Metering, quota, cost and payment-sync logic. It raises domain errors and knows nothing about HTTP.

- **Data layer (app/repositories, app/models).** The only place queries are written. Every method requires a tenant identifier, so cross-tenant reads are impossible by construction rather than by convention.
- **Background job (app/jobs).** Usage rollups and quota alerts at 80% and 100%, with three retries and a recorded failure alert.

4.1 The metering path

A single billable request follows this sequence:

1. Authenticate the tenant from a hashed API key, otherwise return 401.
2. Validate the request body, returning 422 for bad input and never a 500.
3. Check the idempotency key; if it has been seen, return the stored response and record nothing new.
4. Take the per-tenant metering lock, which serialises everything that follows.
5. Check the subscription is in good standing, otherwise return 402.
6. Check that used plus requested is within the limit, otherwise return 429.
7. Price the usage in integer micro-cents.
8. Write the usage event and the idempotency record in one transaction, so both are stored or neither is.

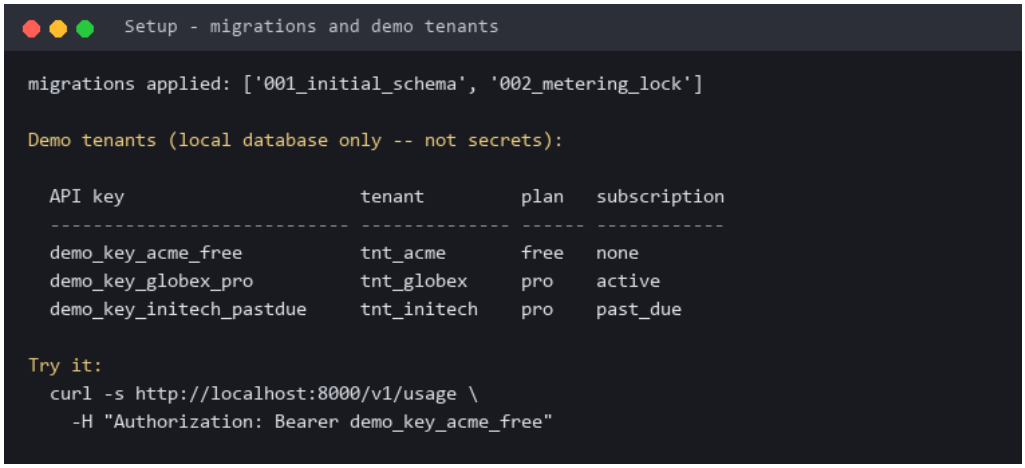

Figure 2 — Applying the database migrations and seeding demonstration tenants. The service starts with one command and needs no external database server.

5. Implementation Highlights

5.1 Exactly-once metering

The guarantee is a unique database index on the tenant and idempotency key together, not application logic. The distinction matters. Checking for a row and then inserting one leaves a window between the two statements in which a second request can do the same thing; a unique constraint has no such window. The service therefore attempts the insert and handles the resulting integrity error, treating the loser of the race as a replay and returning the winner's stored response.

The response of the first attempt is stored in full, so a retry receives an identical answer rather than a newly computed one. A request fingerprint is stored alongside it, so reusing a key with a different payload returns a 409 conflict rather than a misleading success.

5.2 Money as integers

All amounts are stored as integer micro-cents, where one cent is one million micro-cents. Cents alone are too coarse, because a single token costs a small fraction of a cent and rounding each event to whole cents would lose revenue. There is no floating-point column in the schema, and a test asserts that every costing function returns an integer. Division is deferred to the end of each term so that intermediate rounding cannot accumulate, and rounding is always downward, so a fraction is never rounded up against the customer.

5.3 Honest status codes

402 Payment Required is a reserved status code with no standard convention, so the convention used here is written down explicitly in the repository documentation:

- **402 concerns the plan.** The subscription is past due, unpaid or cancelled. Payment must change before any billable request succeeds.
- **429 concerns the allowance.** The plan is in good standing but this month's quota is spent. The response carries a Retry-After header counting down to the actual reset time, rather than an arbitrary delay.
- **402 is evaluated first.** Telling an unpaid customer that they are out of quota would be untrue and would send them to the wrong remedy.

6. Testing and Verification

The brief does not require tests, but in a billing system they are the cheapest possible insurance. The project has 77 automated tests that run from a single command and are deterministic, including a clock that can be controlled so that month-boundary behaviour is testable.

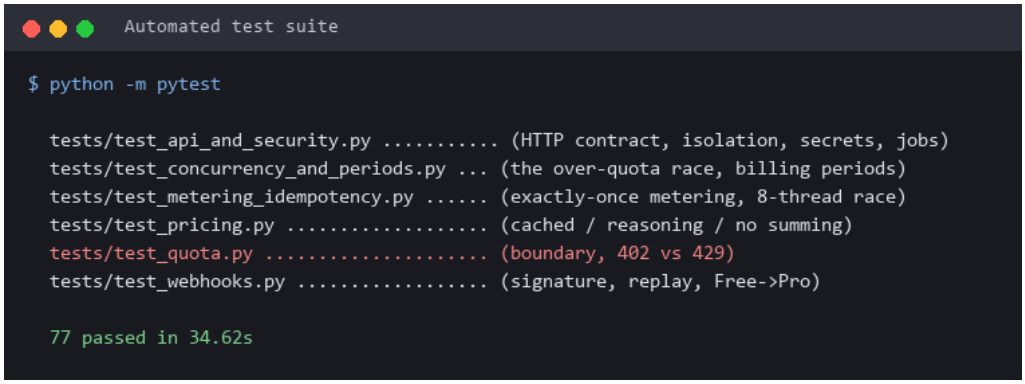

Figure 3 — The full automated test suite.

6.1 Proof: a retried request is billed once

The same request was sent twice with one idempotency key. Both responses carry the same usage event identifier and the same cost; the second is marked as a replay. The database holds a single row.

Proof 1 - the same request sent twice

```
$ curl -X POST localhost:8000/v1/generate \
    -H 'Idempotency-Key: report-demo-001' \
    -d '{"prompt":"summarise this document",
         "simulated_tokens":{"input":1200,"cached_input":800,
                             "output":500,"reasoning":300}}'

--- attempt 1 (first send) ---
{
    "usage_event_id": "ue_32812f05d4044776b7862b80167d63f1",
    "tenant_id": "tnt_acme",
    "event_type": "generate",
    "billed": {
        "api_calls": 1,
        "input_tokens": 1200,
        "cached_input_tokens": 800,
        "output_tokens": 500,
        "reasoning_tokens": 300
    },
    "cost": {
        "micro_cents": 2396000,
        "cents": 2,
        "usd": "0.023960"
    },
    "idempotent_replay": false
}

--- attempt 2 (identical retry, same key) ---
{
    "usage_event_id": "ue_32812f05d4044776b7862b80167d63f1",
    "tenant_id": "tnt_acme",
    "event_type": "generate",
    "billed": {
        "api_calls": 1,
        "input_tokens": 1200,
        "cached_input_tokens": 800,
        "output_tokens": 500,
        "reasoning_tokens": 300
    },
    "cost": {
        "micro_cents": 2396000,
        "cents": 2,
        "usd": "0.023960"
    },
    "idempotent_replay": true
}

$ sqlite3 data/billing.db "SELECT COUNT(*) FROM usage_events"
1

RESULT: 2 requests sent, 1 usage event stored. No double billing.
```

Figure 4 — Two identical requests, one stored usage event. The retry was not billed.

6.2 Proof: the quota boundary is exact

A tenant was driven to precisely its token limit. The request that lands exactly on the limit is allowed; the next single token is refused with 429, a Retry-After header, and a message naming the exact figures involved.

Figure 5 — The limit is inclusive. The customer receives exactly what the plan promises.

6.3 Proof: 402 is distinguished from 429

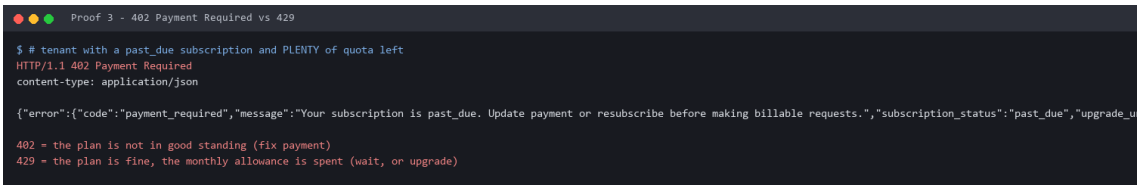

Figure 6 — A tenant with an unpaid subscription and ample quota remaining receives 402, not 429.

6.4 Proof: the money math is correct

The figure below computes the cost of a mixed token bundle by hand and compares it with the value returned by the API. It also shows what the answer would have been had the four token categories been added together and priced once — an error of more than two million micro-cents on a single request.

Figure 7 — Hand-checked pricing. Cached input is charged at its own cheaper rate and reasoning tokens at the output rate.

7. A Defect Found and Fixed

After the test suite was complete and passing, I went looking for what the tests were not covering rather than re-running what already worked. That exercise found a genuine defect, and it is worth reporting honestly because the reasoning error behind it is instructive.

7.1 The defect

Every concurrency test I had written used a single idempotency key. They all therefore proved the same property: that a retried request is metered once. Nothing tested many different requests arriving at the same moment.

I drove a tenant to 999 of 1,000 API calls, leaving room for exactly one more, and fired twelve simultaneous requests, each with its own key. Seven were accepted. The tenant finished at 1,006 of 1,000 — six calls given away free. This is the second failure mode the brief names, and my own design document had asserted that it could not happen.

7.2 Why it happened

The design document claimed the quota check and the write shared a transaction and were therefore safe. A transaction provides atomicity, but not isolation from a concurrent reader. The quota check is a read and the meter is a write, and two requests interleave between them: both read 999, both conclude they have room, and both write. The unique index that makes

idempotency airtight offers no protection here, because there is no duplicate to detect — these are genuinely different requests.

7.3 The fix

A metering lock column was added to the tenant table. Its value is never read; incrementing it as the first statement of the metering transaction takes a write lock that is held until commit — a row lock on PostgreSQL, the database write lock on SQLite. A second request for the same tenant blocks on that statement and therefore reads usage only after the first has committed. Ordering is the entire point: locking after the read would achieve nothing, because the stale read would already have happened.

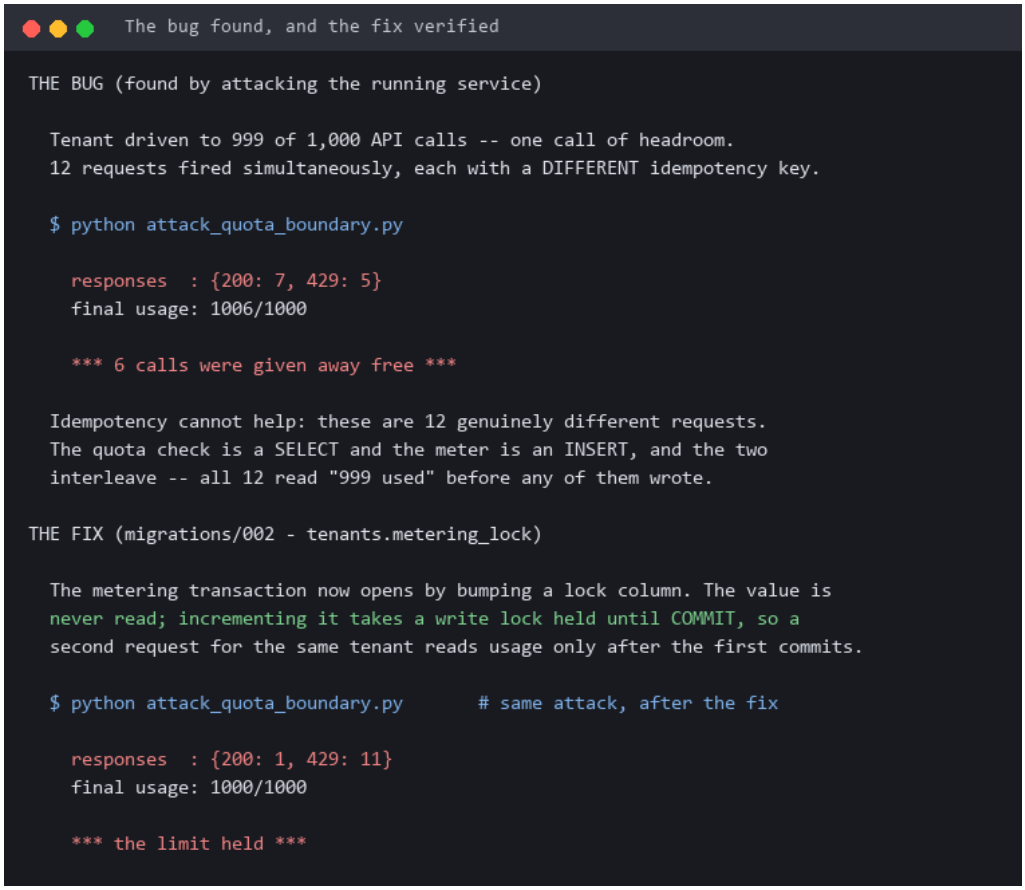

Figure 8 — The same attack before and after the fix. Twelve concurrent requests, one call of headroom.

Two supporting details were needed. The quota gates now roll back on rejection, so a refused request leaves no trace of the lock. SQLite required a busy timeout so that a waiting request waits rather than failing outright, which would have turned a merely contended request into a server error. Six regression tests now cover this and the related boundary cases.

8. Live Stripe Integration

The payment integration was verified end to end against a real Stripe sandbox account, in test mode throughout. No live-mode key was ever used, and the application refuses to start if it is given one.

The complete sequence that was exercised:

9. The service created a genuine Stripe Checkout session through the Stripe API.
10. Payment was completed on Stripe's own hosted page using the documented test card 4242 4242 4242 4242. No real money moves in sandbox mode.
11. Stripe delivered thirteen signed events from that single checkout, forwarded to the local service by the Stripe command-line tool.
12. The checkout completion event moved the tenant from the Free plan to the Pro plan, and the usage endpoint immediately reported the larger limits.

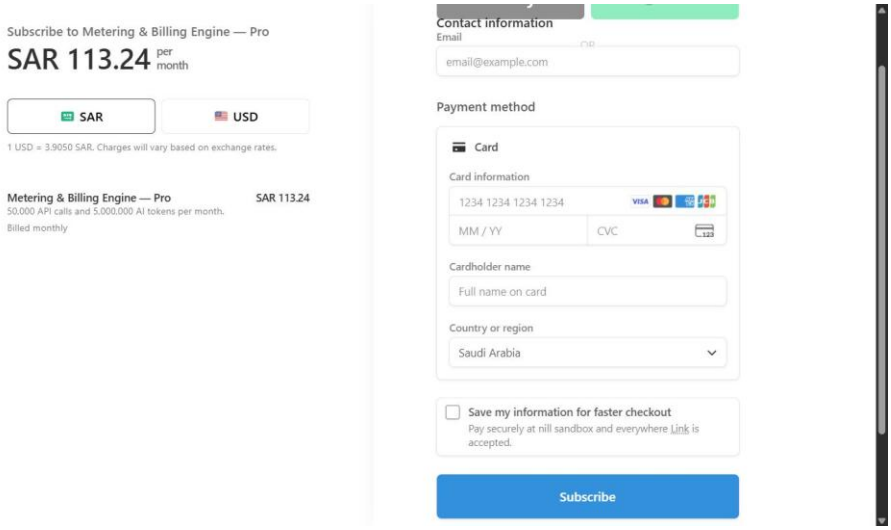

Figure 9 — The Stripe-hosted checkout page generated by the service, showing the Pro plan and its limits.

Figure 10 — The signed events delivered by Stripe, and the tenant's plan before and after.

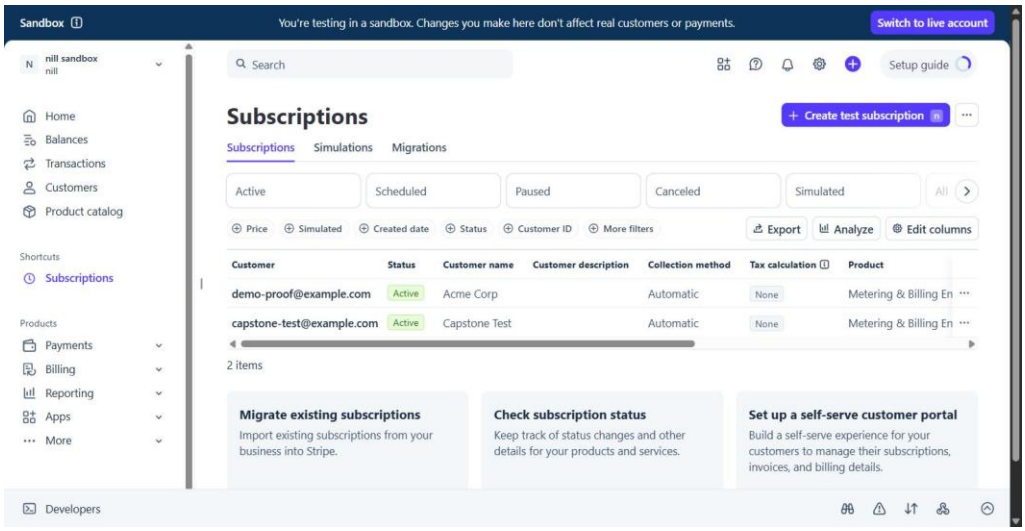

Figure 11 — Active subscriptions in the Stripe sandbox dashboard. The banner confirms that no real customers or payments are affected.

8.1 Webhook security

Payment state is mirrored from Stripe and never inferred from the browser. Two attacks were run against the running service using a real event that Stripe had just delivered.

- **A forged signature** — the same event signed with the wrong secret — was rejected with 400, and nothing was written.
- **A legitimate redelivery** — the same event identifier with a valid signature — returned duplicate_ignored and was processed only once.

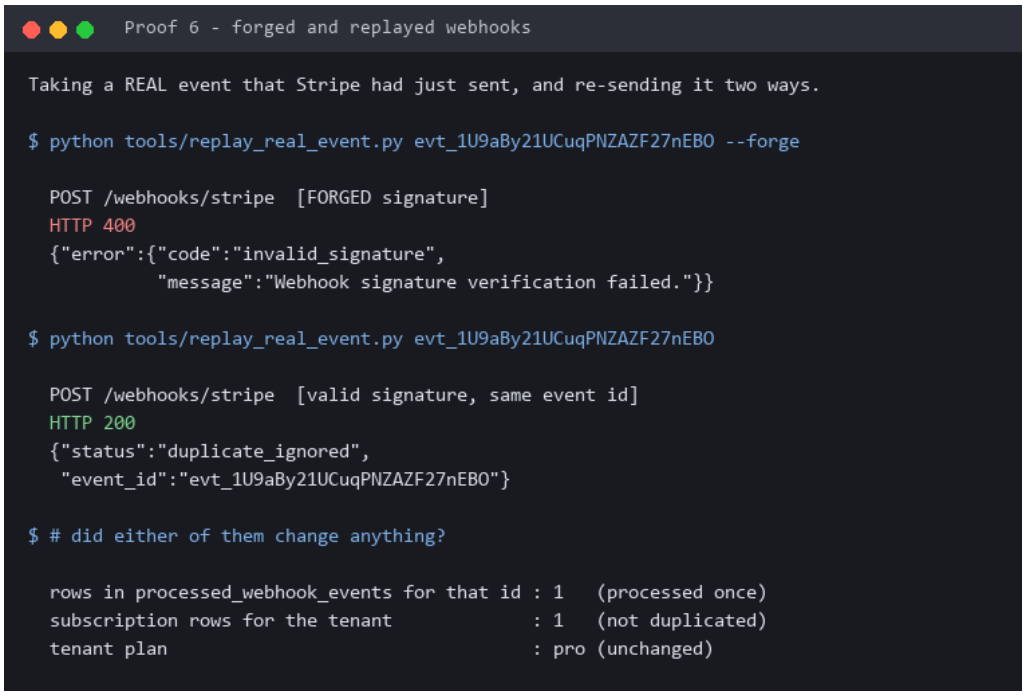

Figure 12 — Forged and replayed webhooks, tested against the real signing secret.

Deduplication keys on the event identifier rather than its timestamp or content, because Stripe does not guarantee ordering and distinct events can share a timestamp. Event types the service does not handle receive a successful response rather than an error, because an error would cause Stripe to retry them for three days.

9. Security and Secret Handling

Repository hygiene was treated as a hard requirement from the first commit rather than something to tidy afterwards.

- **Secrets are excluded before any code exists.** The .env file was added to .gitignore in the very first commit. A full history search confirms it has never been committed.
- **Live mode is impossible.** The application refuses to start if given a live-mode key, so it cannot move real money even by mistake.
- **Secrets cannot leak through logs.** A logging filter redacts anything resembling an API key, webhook secret or bearer token before a log record is written, so a key cannot escape inside an error trace.
- **API keys are stored hashed.** Only the SHA-256 hash of a tenant key is stored; authentication is a hash comparison.

- **Evidence is redacted.** The documentation contains no key material; secrets appear only in the local environment file.

10. Results Against the Requirements

Every requirement in the brief is met, and each has a corresponding proof captured from the running service in the repository's evidence document.

Requirement	Status	Evidence
Exactly one usage event under retries	Met	Figure 4; 8-thread concurrency test
Quota enforced with 429 / 402	Met	Figures 5 and 6
Monthly cost rollup per tenant	Met	Figure 7
Cached, reasoning and output pricing	Met	Figure 7; pricing tests
Checkout works in Stripe test mode	Met	Figures 9 to 11
Webhooks verified and deduplicated	Met	Figure 12
Tenant-isolated data model	Met	Isolation test; migrations
Documentation and submission pack	Met	Repository root

The seven requirements shared by every capstone in the track are also satisfied: a layered architecture, validation at the boundary that never produces a server error from bad input, a background job with retries and a failure alert, schema applied as migrations with tenant isolation, idempotency where it matters, clean secret handling, and per-call cost attribution.

11. Limitations and Future Work

The following limitations are stated plainly in the repository as well, because an honest boundary is more useful to a reader than an overstated claim.

- **Test mode only.** The integration has never been exercised in live mode, and the startup guard prevents it.
- **SQLite by default.** This keeps the project runnable on any machine with no additional software. A PostgreSQL configuration is provided, but has not been load-tested.
- **The scheduler runs in-process.** Two instances of the application would each run the background job. Alerts remain correct because of their unique constraint, but a distributed deployment would need a single scheduler.
- **Calendar-month quota windows.** Quotas reset on the first of the month rather than on each customer's subscription anniversary.

- **Idempotency records are retained.** They are not pruned, because they also serve as the billing audit trail. The table therefore grows with usage.
- **Out of scope by design.** Invoicing, proration and overage billing are listed in the brief as optional extensions and were not attempted, in favour of completing the core correctly.

12. Conclusion

The project delivers a working usage metering and billing engine that meets every requirement in the brief, verified against a real Stripe sandbox rather than a simulation, and covered by 77 automated tests.

The most valuable part of the work was not writing the original implementation but attacking it afterwards. The over-quota defect described in Section 7 had passed every test I had written, because all of those tests examined the same property from slightly different angles. Finding it required treating my own design document as a claim to be disproved rather than a description to be trusted.

That produced the lesson I would carry into professional work: idempotency and quota enforcement look like one problem — preventing something from happening more than it should — and are in fact two problems requiring two different mechanisms. A unique constraint solves the first and cannot touch the second. Recognising where a guarantee genuinely comes from, rather than where it appears to come from, is the difference between a billing system that is correct and one that merely has not failed yet.